D7 Review practice

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track D: Finding errors in code 2
D7 Review practice 3
D7-L1 Review practice: Python and pandas 3
D7-L2 Review practice: PySpark and Spark SQL 6
D7-L3 Review practice: Databricks SQL and DAX 11
Track D best-practice checklist 16
Appendix: solutions and answer keys 17

Track D: Finding errors in code

Find and fix errors fast, including errors in code that Claude and Copilot write.

D7 Review practice

Roadmap: Every month. 25 AI-style snippets with seeded bugs and a hidden answer key.

D7-L1 Review practice: Python and pandas

Concept

This lesson is practice. Each exercise shows a short snippet in the style an assistant writes: tidy, plausible and typed. Each holds one seeded bug, drawn from your reference pipeline and from the lessons in Track D. The answer key stays hidden until you submit.

How to review each snippet:

1. Read the prompt's goal first. The bug is a gap between the goal and the code.
2. Walk the lines in order. For each, ask: what does this assume, and what input breaks it?
3. Tap the line you would change. Then write the fix, and run it against the checker.

What to look for in Python and pandas code:

- Names used but never imported, and imports that shadow built-ins.
- Deprecated calls: `datetime.utcnow()`, chained assignment, `applymap`.
- Type hints that lie: `str = None`.
- Truthiness where 0, 0.0 or an empty string is a valid value.
- Mutable default arguments.
- Joins that drop or duplicate rows, and nulls that behave unexpectedly in filters.
- Text compared where numbers were meant, such as wave ids.

Your score counts two things: bugs found (the right line) and fixes proposed (a fix that passes the checker). Aim to find the bug before you open a hint. When a hint helps, note which one: that is the check you skipped.

Work at your own pace. A snippet you get wrong is worth more than one you get right, as long as you read the explanation.

Why it matters

Reviewing is a separate skill from writing. Engineers who review AI output well do it by habit: goal first, then line by line, asking what each line assumes. Short, deliberate practice on seeded bugs builds that habit faster than waiting for real bugs to appear in production.

Worked example: weak code

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave."""
1     return max(wave_ids)
```

1. **Line 3.** Text comparison: max(['W9', 'W10']) is 'W9'.

Worked example: strong code

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave. Wave ids look like 'W' plus a
    number."""
1     return max(wave_ids, key=lambda w: int(w[1:]))
```

1. **Line 3.** Compares the numbers, so W10 beats W9.

Goal first, then line by line: what does each line assume, and which input breaks it?

Recap card

- **Review routine for a snippet** Goal first, then each line: what does it assume, and which input breaks it?
- **Python bugs seeded in this lesson** Missing import, mutable default, utcnow, str = None, truthiness of 0.0, inner join, text max.
- **When truthiness is fine** Testing a list or dict for emptiness. Not for numbers where 0 is valid.

Practice

Exercise 1. Spot the bug (level 1)

Goal: return a new unique run id for the run log. Find the bug and fix it.

```
1 def new_run_id() -> str:
2     """Return a new unique run id."""
3     return str(uuid.uuid4())
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Predict the output (level 1)

Goal: each call starts with an empty list of warnings. What does this print?

```python
def add_warning(msg, warnings=[]):
    warnings.append(msg)
    return warnings

add_warning("weight sum low")
print(len(add_warning("null ids")))
```

Exercise 3. Spot the bug (level 2)

Goal: record the run time as an aware UTC timestamp. Find the bug and fix it.

```python
from datetime import datetime, timezone

def run_timestamp() -> datetime:
    """Current time in UTC for the run log."""
    return datetime.utcnow()
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Spot the bug (level 2)

Goal: `error_message` is optional. The signature must say so for mypy. Find the bug and fix the signature.

```python
def log_run(
    pipeline_name: str,
    status: str,
    error_message: str = None,
) -> None:
    """Append one record to the run log."""
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Spot the bug (level 3)

Goal: count respondents with a recorded weight. A weight of 0.0 is recorded and valid for this count. Find the bug and fix it.

```python
def count_weighted(weights: list[float | None]) -> int:
    """Number of respondents with a recorded weight."""
    recorded = [w for w in weights if w]
    return len(recorded)
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Multiple choice (level 3)

Goal: keep every respondent except those in region `N`, including respondents with no region. Does `df[df["region"] != "N"]` meet the goal in pandas 3?

- a) Yes: in pandas, != keeps missing regions
- b) No: missing regions are dropped
- c) No: it raises on missing values

- d) Only if the column is a string dtype

Why?

- r1) pandas treats missing != 'N' as True, so those rows stay. Spark would drop them.
- r2) pandas fills missing values with empty strings before comparing.

Exercise 7. Spot the bug (level 4)

Goal: attach region names to every respondent. Respondents whose code is missing from the lookup must stay, with no name. Find the bug and fix it.

```
1 import pandas as pd
2
3 def add_region(resp: pd.DataFrame, regions: pd.DataFrame) -> pd.DataFrame:
4     """Attach region names. One row per respondent."""
5     lookup = regions.dropna(subset=["code"])
6     return resp.merge(lookup, on="code", how="inner",
    validate="many_to_one")
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Spot the bug (level 5)

Goal: return the most recent wave from ids like `W9`, `W10`, `W11`. Find the bug and fix it.

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave id."""
3     if not wave_ids:
4         raise ValueError("no waves")
5     return max(wave_ids)
```

Write down the line number of the bug. Then write the fix.

Sources

- Snippet behaviours (NameError without import, mutable default sharing, utcnow naive and deprecated, pandas != keeps missing, max of strings) match local runs. (verified: Local runs on CPython 3.13.12 and pandas 3.0.2, September 2026 (see D1, D3 and D5 claims).)

D7-L2 Review practice: PySpark and Spark SQL

Concept

Nine snippets of PySpark and Spark SQL, each with one seeded bug. Most come straight from your reference pipeline's known defects. The answer key stays hidden until you submit.

Use the same routine as lesson D7-L1: read the goal, walk the lines, ask what each line assumes. Spark adds its own questions:

- **Imports.** Is `sum`, `min`, `max`, `round` or `filter` imported by name from `pyspark.sql.functions`? Is `functions as F` used instead?
- **Filters.** Does a comparison with a nullable column drop nulls? Is a boolean compared with `== True`?
- **Grain.** Does every `groupBy` include each grain column the docstring names? Does every join keep the grain?
- **Aggregates.** Can a `sum(when(...))` return null? Is a division guarded?
- **Passes over the data.** Are there several `count()` calls on an uncached DataFrame, where one `agg` would do?
- **Writes.** Does a MERGE source hold one row per key? Is a rename silently skipped when the source column is missing?
- **SQL.** Does `NOT IN` meet a nullable subquery?

For each snippet, tap the line, then write the fix. The checker tests the fix statically: it looks for the pattern the standard requires. Your score counts bugs found and fixes proposed.

When you finish, compare your list of misses with the checklist above. The item you missed most is the one to add to your personal review routine.

Why it matters

Spark code fails quietly more often than Python code. A shadowed import fails loudly. A missing grain column, a null sum or a duplicate MERGE key can pass every run and still publish wrong numbers. Practice on the defects from your own pipeline trains your eye for the ones that will appear in its next version.

Worked example: weak code

```
1 from pyspark.sql.functions import col, sum
2
3 metrics = silver.filter(col("in_scope") ==
      True).groupBy("wave_id").agg(sum("final_weight"))
```

1. **Line 1.** Shadows Python's sum for the whole module.
2. **Line 3.** `== True` trips ruff E712. Filter on the column directly.

Worked example: strong code

```
1 from pyspark.sql import functions as F
2
```

```
❷ metrics =
      silver.filter(F.col("in_scope")).groupBy("wave_id").agg(F.sum("final_weight"))
```

1. **Line 1.** Python built-ins keep their meaning.
2. **Line 3.** A boolean column is already a condition.

For Spark code, check imports, null behaviour in filters and sums, grain in groupBy and joins, passes over data and MERGE keys.

Recap card

- **Spark review checklist** Imports, null behaviour in filters and sums, grain in groupBy, passes over data, MERGE keys, silent renames, NOT IN.
- **MERGE with repeated source keys** Fails when several source rows match one target row. Deduplicate the source on the merge keys first.
- **withColumnRenamed on a missing column** A silent no-op. Check df.columns and raise.

Practice

Exercise 1. Spot the bug (level 1)

Goal: a gold aggregation module that also uses Python's built-in `sum` in a helper. Find the bug and fix the import.

```
1 from pyspark.sql.functions import col, count, sum, when
2
3 def total_checks(results: list[dict[str, str]]) -> int:
4     """Number of passed checks."""
5     return sum(1 for r in results if r["status"] == "PASS")
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Spot the bug (level 1)

Goal: keep in-scope respondents. Find the line that breaks the standard and fix it.

```
1 from pyspark.sql import functions as F
2
3 in_scope = silver.filter(F.col("in_scope") == True)
4 in_scope.write.mode("overwrite").saveAsTable("research_dev.nat_in_scope")
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

Goal: drop test panel respondents and keep everyone else, including respondents with no panel recorded. Find the bug and fix it.

```
1 from pyspark.sql import functions as F
```

```
2
3 live = respondents.filter(F.col("panel_type") != "test")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Multiple choice (level 2)

Goal: gold shows 0 unweighted reach for a subgroup where nobody was reached. The reference code uses `sum(when(col("reach_flag") == 1, 1)).alias("unweighted_reach")`. What does gold show?

- a) NULL
- b) 0
- c) The subgroup's row count
- d) An error

Why?

- r1) when without otherwise is null for non-matching rows, and a sum of nulls is null.
- r2) Spark drops zero sums.

Exercise 5. Spot the bug (level 3)

Goal (from the docstring): reach metrics at tracker x wave x subgroup grain. Find the bug and fix it.

```
1 from pyspark.sql import functions as F
2
3 metrics = (
4     silver.filter(F.col("in_scope"))
5     .groupBy("wave_id", "age_group", "gender", "region")
6     .agg(F.sum("final_weight").alias("weighted_base"))
7 )
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Spot the bug (level 3)

Goal: table health counts for the run log, computed efficiently. Find the line that shows the problem and rewrite the counts as one aggregation.

```
1 from pyspark.sql import functions as F
2
3 df = spark.table(table_name)
4 health = {
5     "row_count": df.count(),
6     "null_ids": df.filter(F.col("respondent_id").isNull()).count(),
7     "null_weights": df.filter(F.col("final_weight").isNull()).count(),
8 }
```

Write down the line number of the bug. Then write the fix.

Exercise 7. Spot the bug (level 4)

Goal: MERGE a redelivery into silver on respondent and wave. Redeliveries sometimes repeat a respondent. Find the bug and fix it.

```python
from delta.tables import DeltaTable

target = DeltaTable.forName(spark, "research_silver.nat_respondents")
(target.alias("t")
    .merge(delivery.alias("s"), "t.respondent_id = s.respondent_id AND
   t.wave_id = s.wave_id")
    .whenMatchedUpdateAll()
    .whenNotMatchedInsertAll()
    .execute())
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Predict the output (level 4)

Goal: list respondents outside the excluded regions. `excluded` holds `'N'` and one NULL. How many rows does this return from 1,000 respondents, 300 of them in N?

```sql
SELECT COUNT(*) FROM respondents
WHERE region_code NOT IN (SELECT code FROM excluded);
```

Exercise 9. Spot the bug (level 5)

Goal: rename delivery columns to canonical names. If a source column is missing, stop with an error naming it. Find the bug and fix it.

```python
def rename_columns(bronze_df, column_map: dict[str, str]):
    """Rename source columns to canonical names."""
    for src, tgt in column_map.items():
        bronze_df = bronze_df.withColumnRenamed(src, tgt)
    return bronze_df
```

Write down the line number of the bug. Then write the fix.

Sources

- Snippet behaviours (shadowed sum NOT_COLUMN_OR_STR, != drops nulls, sum(when) null, NOT IN with NULL returns no rows, withColumnRenamed no-op, MERGE multiple-source-rows failure) match local runs. (verified: Local runs on PySpark 4.0.4 and delta-spark 4.0.1 (see D1, D3, D4 claims and practice/tests/test_a3_l1_merge.py), September 2026.)

D7-L3 Review practice: Databricks SQL and DAX

Concept

Eight snippets of Databricks SQL and DAX, each with one seeded bug. SQL and DAX look simple, which makes their bugs easy to read past. The answer key stays hidden until you submit.

What to look for in Databricks SQL:

- **Syntax from another platform.** `CREATE ROW ACCESS POLICY` and `CREATE MASKING POLICY` are Snowflake. Databricks uses a SQL UDF with `SET ROW FILTER` or `SET MASK`.
- **Grants to individuals.** Your standard grants to groups, never to one user's email.
- **Group checks.** `is_member()` checks workspace groups. Unity Catalog guidance favours `is_account_group_member()`.
- **Names.** Two-part names resolve against the current catalog. Use three-level names: `catalog.schema.table`.
- **Ratios.** An `AVG` of percentages weights every subgroup equally. Recompute ratios from their sums.
- **MERGE sources.** One row per key, or the statement fails.

What to look for in DAX:

- **Division.** `/` returns infinity or an error on a zero denominator. `DIVIDE` returns BLANK.
- **Filter context.** `ALL(fact_table)` removes every filter on the table, wave and region slicers included. Name the one dimension whose filter you mean to remove.

Review each snippet with the same routine: goal, then line by line. For SQL, also check each statement against the Databricks SQL reference in your head: does this statement exist here? The checker tests your fix against the pattern the standard requires. Your score counts bugs found and fixes proposed.

This lesson closes the practice module. The capstone review in Phase 6 uses the same skills on a whole pull request.

Why it matters

Security SQL and report measures carry outsized risk. A grant to one person survives their move to another team. A row filter written in the wrong dialect

leaves a table open. A DAX measure with the wrong filter context shows a plausible number to every viewer of a dashboard. Reviewing these few lines carefully protects many readers.

Worked example: weak code

```
(1) SELECT region, AVG(weighted_reach_pct) AS reach_pct
 2  FROM research_gold.nat_metrics
 3  GROUP BY region;
```

1. **Line 1.** An average of subgroup percentages. A subgroup of 12 counts as much as one of 1,200.

Worked example: strong code

```
 1  SELECT region,
(1)         SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
 3  FROM research_gold.nat_metrics
 4  GROUP BY region;
```

1. **Line 2.** Total over total, with the zero base guarded.

For SQL, check the dialect, grants, group functions, names and ratios. For DAX, check DIVIDE and the exact filter each ALL removes.

Recap card

- **SQL review checklist** Dialect, grants to groups, is_account_group_member, three-level names, ratios from sums, one source row per MERGE key.
- **DAX review checklist** DIVIDE for every ratio. ALL on the one dimension you mean, never the whole fact table.
- **Deduplicating a MERGE source in SQL** ROW_NUMBER() OVER (PARTITION BY keys ORDER BY ingestion_timestamp DESC), keep rn = 1.

Practice

Exercise 1. Spot the bug (level 1)

Goal: restrict `research_silver.nat_respondents` rows by region with a Databricks row filter. Find the bug and fix it.

```
 1  CREATE FUNCTION research_governance.region_filter(region STRING)
 2    RETURN is_account_group_member(concat('analysts_', region));
 3  CREATE ROW ACCESS POLICY region_policy ON research_silver.nat_respondents
      (region);
```

Write down the line number of the bug. Then write the fix.

Exercise 2. Spot the bug (level 1)

Goal: let the analysts read the gold metrics table, following the standard. Find the bug and fix it.

```
GRANT USE CATALOG ON CATALOG research TO `analysts`;
GRANT USE SCHEMA ON SCHEMA research.research_gold TO `analysts`;
GRANT SELECT ON TABLE research.research_gold.nat_metrics TO
    `ana.lopez@example.com`;
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

Goal: a column mask that shows postcodes only to members of the account-level group `analysts`, as Unity Catalog guidance recommends. Find the bug and fix it.

```
CREATE FUNCTION research_governance.mask_postcode(p STRING)
  RETURN CASE WHEN is_member('analysts') THEN p ELSE '***' END;
ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Multiple choice (level 2)

Goal: a scheduled query must read production gold. It runs `SELECT * FROM research_gold.nat_metrics`. The job's session has `research_dev` as its current catalog. What does it read?

- a) research_dev.research_gold.nat_metrics, if it exists
- b) research.research_gold.nat_metrics, because gold is production
- c) An error: two-part names are not allowed
- d) Both tables, unioned

Why?

- r1) A two-part name resolves against the current catalog. Only a three-level name is unambiguous.
- r2) Databricks always prefers dev catalogs.

Exercise 5. Spot the bug (level 3)

Goal: weighted reach percentage per region from the gold table, which has one row per subgroup with `weighted_reach` and `weighted_base`. Find the bug and fix it, guarding a zero base.

```
SELECT region,
       AVG(weighted_reach / weighted_base) AS reach_pct
FROM research.research_gold.nat_metrics
```

```
4 GROUP BY region;
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Fill the gap (level 3)

Goal: a reach percentage measure that shows an empty cell, not infinity, where the base is zero. The assistant wrote `RETURN reach_w / base_w`. Complete the fix.

```
1 Reach % =
2 VAR reach_w = SUM ( fact_reach[weighted_reach] )
3 VAR base_w = SUM ( fact_reach[weighted_base] )
4 RETURN _____(1) ( reach_w, _____(2) )
```

Exercise 7. Spot the bug (level 4)

Goal: each brand's share of weighted reach across all brands, respecting the wave and region slicers. Find the bug and fix it.

```
1 Brand share =
2 VAR this_brand = SUM ( fact_reach[weighted_reach] )
3 VAR all_brands = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
     ( fact_reach ) )
4 RETURN DIVIDE ( this_brand, all_brands )
```

Write down the line number of the bug. Then write the fix.

Exercise 8. Spot the bug (level 5)

Goal: MERGE a redelivery into silver. The delivery can repeat a respondent within a wave. Keep the row with the latest `ingestion_timestamp`. Find the bug and fix it.

What you need to know.

`MERGE INTO target AS t USING source AS s ON <condition>` is the SQL form of the Delta merge from lesson A3-L1. Each target row is matched against the source rows that meet the `ON` condition:

- `WHEN MATCHED THEN UPDATE SET *` overwrites the matched target row with the source row's columns.
- `WHEN NOT MATCHED THEN INSERT *` inserts source rows with no match.

The `USING` part can be a table or a subquery in brackets. In Databricks SQL, `SELECT * EXCEPT (col)` returns every column except `col`.

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_raw_w14 AS s
3 ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Write down the line number of the bug. Then write the fix.

Sources

- Databricks row filters and masks, is_account_group_member, three-level namespace and grants follow the docs verified in Track A. (verified: https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/manually-apply and related pages (checked 2026-09-25))
- SELECT * EXCEPT (col) is supported in Databricks SQL. (verified: https://docs.databricks.com/aws/en/sql/language-manual/sql-ref-syntax-qry-star (checked 2026-09-25))

Track D best-practice checklist

☐ Read a traceback from the bottom up. Find the first frame in your own code.

☐ Reproduce the error on the smallest data that shows it.

☐ Check row counts before and after every join.

☐ Test numbers with `is None`, never with truthiness, when zero is a valid value.

☐ Never use a mutable object as a default argument.

☐ Check every function and parameter an AI suggests against the official docs for your version.

☐ Run AI code on a small input with a known answer before you trust it.

☐ Write the failing test first. A test that only repeats the code proves nothing.

☐ Ask the model to list its assumptions. Check each one.

☐ State versions, constraints, expected outputs and tests in every prompt.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

D7-L1 Review practice: Python and pandas

D7-L1 exercise 1

Hints. (1) Where does uuid come from? (2) It is never imported. (3) Add import uuid.

Answer. The bug is on line 3.

Fixed version:

```
1 import uuid
2
3
4 def new_run_id() -> str:
5     """Return a new unique run id."""
6     return str(uuid.uuid4())
```

Reference defect 1. NameError on the first run. ruff rule F821 flags undefined names before you run anything.

Common wrong answers.

- Line 1: The signature is fine. The body uses a name that was never imported.

D7-L1 exercise 2

Hints. (1) When is the default list created? (2) Once, at definition. (3) Both calls share it.

Answer.

```
1 2
```

Mutable default argument. Default to None and create the list inside.

Common wrong answers.

- `1`: The default list is created once and shared, so the second call sees the first call's warning.

D7-L1 exercise 3

Hints. (1) Is the returned datetime aware? (2) utcnow() returns a naive one, and is deprecated. (3) datetime.now(timezone.utc).

Answer. The bug is on line 5.

Fixed version:

```
1 from datetime import datetime, timezone
2
3 def run_timestamp() -> datetime:
4     """Current time in UTC for the run log."""
5     return datetime.now(timezone.utc)
```

Reference defect 6. Naive timestamps cannot be compared with aware ones and hide their zone.

Common wrong answers.

- Line 1: The import is right, and timezone is even imported already. It is just not used.

D7-L1 exercise 4

Hints. (1) Can error_message be None? (2) Then the type must allow None. (3) str | None = None.

Answer. The bug is on line 4.

Fixed version:

```
1 def log_run(
2     pipeline_name: str,
3     status: str,
4     error_message: str | None = None,
5 ) -> None:
6     """Append one record to the run log."""
```

Reference defect 7. `str = None` claims the value is always a string. mypy --strict rejects it.

Common wrong answers.

- Line 3: status is required and always a string. The optional parameter is the next one.

D7-L1 exercise 5

Hints. (1) Is 0.0 truthy? (2) No. (3) Use `is not None`.

Answer. The bug is on line 3.

Fixed version:

```
1 def count_weighted(weights: list[float | None]) -> int:
2     """Number of respondents with a recorded weight."""
3     recorded = [w for w in weights if w is not None]
4     return len(recorded)
```

Truthiness treats 0.0 like None. Your standard: test for None with `is None`.

Common wrong answers.

- Line 4: len is right. The filter above drops 0.0.

D7-L1 exercise 6

Hints. (1) Lesson D3-L1: nulls in filters. (2) pandas and Spark differ here. (3) pandas keeps them.

Answer. a) Yes: in pandas, != keeps missing regions Because: r1) pandas treats missing != 'N' as True, so those rows stay. Spark would drop them.

The same filter in PySpark would drop the null rows. Know which engine you are reviewing.

Common wrong answers.

- `b`: That is Spark and SQL behaviour. pandas keeps them.
- `c`: The comparison runs without error.
- `d`: The result is the same for object and str dtypes.
- `a|r2`: Right answer, wrong reason. pandas does not fill. Missing compares as not-equal.

D7-L1 exercise 7

Hints. (1) Which join type keeps unmatched left rows? (2) Inner drops them. (3) how='left'.

Answer. The bug is on line 6.

Fixed version:

```
1 import pandas as pd
2
3 def add_region(resp: pd.DataFrame, regions: pd.DataFrame) -> pd.DataFrame:
4     """Attach region names. One row per respondent."""
5     lookup = regions.dropna(subset=["code"])
6     return resp.merge(lookup, on="code", how="left",
    validate="many_to_one")
```

An inner join quietly removes respondents with unknown codes. The grain promise, one row per respondent, needs a left join.

Common wrong answers.

- Line 5: Dropping null lookup keys is right: pandas would otherwise match null to null.

D7-L1 exercise 8

Hints. (1) What is max(['W9', 'W10'])? (2) 'W9': text comparison. (3) Use key=lambda w: int(w[1:]).

Answer. The bug is on line 5.

Fixed version:

```
1 def latest_wave(wave_ids: list[str]) -> str:
2     """Return the most recent wave id."""
3     if not wave_ids:
4         raise ValueError("no waves")
5     return max(wave_ids, key=lambda w: int(w[1:]))
```

Text sorts character by character. The truthiness test on line 3 is fine: an empty list is exactly what it should catch.

Common wrong answers.

- Line 3: `if not wave_ids` is correct for a list: an empty list is the case to catch.

D7-L2 Review practice: PySpark and Spark SQL

D7-L2 exercise 1

Hints. (1) Which sum does line 5 call? (2) The one imported on line 1. (3) Import functions as F.

Answer. The bug is on line 1.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 def total_checks(results: list[dict[str, str]]) -> int:
4     """Number of passed checks."""
5     return sum(1 for r in results if r["status"] == "PASS")
```

Reference defects 2 and 3. The helper raises NOT_COLUMN_OR_STR because it calls Spark's sum on a generator.

Common wrong answers.

- Line 5: This line is correct Python. It fails only because line 1 replaced sum.

D7-L2 exercise 2

Hints. (1) What does ruff E712 flag? (2) Comparison to True. (3) The column is already boolean.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
```

```
3 in_scope = silver.filter(F.col("in_scope"))
4 in_scope.write.mode("overwrite").saveAsTable("research_dev.nat_in_scope")
```

Reference defect 12. `F.col("in_scope")` is the condition.

Common wrong answers.

- Line 4: Overwriting a dev table is fine here. The standard issue is the comparison.

D7-L2 exercise 3

Hints. (1) What is null != 'test' in Spark? (2) Null, which the filter drops. (3) eqNullSafe.

Answer. The bug is on line 3.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 live = respondents.filter(~F.col("panel_type").eqNullSafe("test"))
```

Spark's three-valued logic drops null rows from a != filter.

Common wrong answers.

- Line 1: The import follows the standard.

D7-L2 exercise 4

Hints. (1) What does when() give for non-matching rows? (2) Null. (3) Sum of nulls.

Answer. a) NULL Because: r1) when without otherwise is null for non-matching rows, and a sum of nulls is null.

Reference defect 16. Wrap with `F.coalesce(..., F.lit(0))`.

Common wrong answers.

- `b`: That is the goal. It needs coalesce(..., lit(0)) or otherwise(0).
- `c`: Non-matching rows contribute null, not 1.
- `d`: No error. The null passes silently.
- `a|r2`: Right answer, wrong reason. A sum of real zeros is 0. A sum of only nulls is null.

D7-L2 exercise 5

Hints. (1) Compare the groupBy with the stated grain. (2) Which grain column is missing? (3) tracker_id.

Answer. The bug is on line 5.

Fixed version:

```
from pyspark.sql import functions as F

metrics = (
    silver.filter(F.col("in_scope"))
    .groupBy("tracker_id", "wave_id", "age_group", "gender", "region")
    .agg(F.sum("final_weight").alias("weighted_base"))
)
```

Reference finding 15. Without tracker_id, two trackers' respondents merge into one row per wave and subgroup.

Common wrong answers.

- Line 4: The in-scope filter follows the standard.
- Line 6: The aggregate is fine. The grouping misses a grain column.

D7-L2 exercise 6

Hints. (1) How many times is the table scanned? (2) Once per count(). (3) One agg with conditional sums.

Answer. The bug is on line 5, 6, 7.

Fixed version:

```
from pyspark.sql import functions as F

df = spark.table(table_name)
row = df.agg(
    F.count("*").alias("row_count"),
    F.sum(F.col("respondent_id").isNull().cast("int")).alias("null_ids"),

    F.sum(F.col("final_weight").isNull().cast("int")).alias("null_weights"),
).first()
health = row.asDict()
```

Reference defect 11. Three actions mean three scans of an uncached table. One `agg` does it in one.

Common wrong answers.

- Line 3: Reading the table once is right. The three actions below each scan it again.

D7-L2 exercise 7

Hints. (1) What happens when two source rows match one target row? (2) MERGE fails. (3) Deduplicate the source on the keys.

Answer. The bug is on line 5.

Fixed version:

```
1 from delta.tables import DeltaTable
2
3 source = delivery.dropDuplicates(["respondent_id", "wave_id"])
4 target = DeltaTable.forName(spark, "research_silver.nat_respondents")
5 (target.alias("t")
6     .merge(source.alias("s"), "t.respondent_id = s.respondent_id AND
    t.wave_id = s.wave_id")
7     .whenMatchedUpdateAll()
8     .whenNotMatchedInsertAll()
9     .execute())
```

Reference finding 14. Delta MERGE fails when several source rows match one target row. Deduplicate first, ideally keeping the latest delivery.

Common wrong answers.

- Line 6: Updating matched rows is right. The failure comes from two source rows matching one target row.

D7-L2 exercise 8

Hints. (1) Expand NOT IN into AND terms. (2) x <> NULL is null. (3) No row passes.

Answer.

```
1 0
```

Use `NOT EXISTS` or a `left_anti` join when the list can hold nulls.

Common wrong answers.

- `700`: That is what NOT EXISTS returns. NOT IN with a NULL in the list is never true.

D7-L2 exercise 9

Hints. (1) What does withColumnRenamed do with a missing column? (2) Nothing, silently. (3) Check bronze_df.columns first.

Answer. The bug is on line 4.

Fixed version:

```
1 def rename_columns(bronze_df, column_map: dict[str, str]):
2
    """Rename source columns to canonical names. Raises if a source column
    is missing."""
3     missing = sorted(set(column_map) - set(bronze_df.columns))
4     if missing:
5         raise ValueError(f"delivery is missing columns: {missing}")
6     for src, tgt in column_map.items():
7         bronze_df = bronze_df.withColumnRenamed(src, tgt)
8     return bronze_df
```

Reference finding 18. A renamed column in a delivery vanishes without error, and every later step sees a null or a missing field.

Common wrong answers.

- Line 3: Looping over the map is fine. The rename inside it hides a missing column.

D7-L3 Review practice: Databricks SQL and DAX

D7-L3 exercise 1

Hints. (1) Does line 3 exist in Databricks SQL? (2) It is Snowflake syntax. (3) ALTER TABLE ... SET ROW FILTER.

Answer. The bug is on line 3.

Fixed version:

```
1 CREATE FUNCTION research_governance.region_filter(region STRING)
2   RETURN is_account_group_member(concat('analysts_', region));
3 ALTER TABLE research_silver.nat_respondents SET ROW FILTER
     research_governance.region_filter ON (region);
```

Reference defect 4. The UDF is right. The binding statement comes from another platform.

Common wrong answers.

- Line 1: A SQL UDF is exactly what Databricks row filters use.

D7-L3 exercise 2

Hints. (1) Who receives each grant? (2) One line names a person. (3) Grant to the group.

Answer. The bug is on line 3.

Fixed version:

```
1 GRANT USE CATALOG ON CATALOG research TO `analysts`;
2 GRANT USE SCHEMA ON SCHEMA research.research_gold TO `analysts`;
3 GRANT SELECT ON TABLE research.research_gold.nat_metrics TO `analysts`;
```

Grants to groups survive team changes. A grant to one email is forgotten when that person moves.

Common wrong answers.

- Line 1: USE CATALOG to the group is needed and correct.

D7-L3 exercise 3

Hints. (1) Which group function does the goal name? (2) Account-level groups. (3) is_account_group_member.

Answer. The bug is on line 2.

Fixed version:

```
1 CREATE FUNCTION research_governance.mask_postcode(p STRING)
2   RETURN CASE WHEN is_account_group_member('analysts') THEN p ELSE '***'
    END;
3 ALTER TABLE research_silver.nat_respondents ALTER COLUMN postcode SET
    MASK research_governance.mask_postcode;
```

Reference finding 22. is_member checks workspace-local groups. Account groups are the ones Unity Catalog uses.

Common wrong answers.

- Line 3: ALTER COLUMN ... SET MASK is the correct Databricks statement.

D7-L3 exercise 4

Hints. (1) How many parts does the name have? (2) Which part is missing? (3) The current catalog fills it in.

Answer. a) research_dev.research_gold.nat_metrics, if it exists Because: r1) A two-part name resolves against the current catalog. Only a three-level name is unambiguous.

Use three-level names in jobs: `research.research_gold.nat_metrics`.

Common wrong answers.

- `b`: The name does not say which catalog. The session decides.
- `c`: Two-part names work. That is why they are risky.
- `d`: One table is resolved.
- `a|r2`: Right answer, wrong reason. The current catalog decides, whatever it is.

D7-L3 exercise 5

Hints. (1) Does each subgroup weigh the same in AVG? (2) Should it? (3) Total over total.

Answer. The bug is on line 2.

Fixed version:

```
1 SELECT region,
2        SUM(weighted_reach) / NULLIF(SUM(weighted_base), 0) AS reach_pct
3 FROM research.research_gold.nat_metrics
4 GROUP BY region;
```

Recompute ratios from sums at the level you report. NULLIF turns a zero base into a null result instead of an error.

Common wrong answers.

- Line 4: Grouping by region matches the goal.

D7-L3 exercise 6

Hints. (1) What does / return when base_w is 0? (2) Infinity or an error. (3) `DIVIDE` with the base as its second argument.

Answer.

```
1 Reach % =
2 VAR reach_w = SUM ( fact_reach[weighted_reach] )
3 VAR base_w = SUM ( fact_reach[weighted_base] )
4 RETURN DIVIDE ( reach_w, base_w )
```

DIVIDE returns BLANK on a zero or blank denominator, so the cell stays empty (lesson C6-L2).

Common wrong answers.

- g1 = `IFERROR`: IFERROR hides every error, not only a zero base. DIVIDE returns BLANK for a zero or blank denominator.

D7-L3 exercise 7

Hints. (1) Which filters does ALL(fact_reach) remove? (2) All of them. (3) Name the brand dimension.

Answer. The bug is on line 3.

Fixed version:

```
1 Brand share =
2 VAR this_brand = SUM ( fact_reach[weighted_reach] )
3 VAR all_brands = CALCULATE ( SUM ( fact_reach[weighted_reach] ), ALL
     ( dim_brand ) )
4 RETURN DIVIDE ( this_brand, all_brands )
```

The denominator must keep the wave and region filters and drop only the brand filter.

Common wrong answers.

- Line 4: DIVIDE is right. The denominator's context is wrong.

D7-L3 exercise 8

Hints. (1) Can the source hold two rows for one key? (2) Yes, and MERGE fails then. (3) Deduplicate in the USING clause with ROW_NUMBER.

Answer. The bug is on line 2.

Fixed version:

```
MERGE INTO research.research_silver.nat_respondents AS t
USING (
  SELECT * EXCEPT (rn) FROM (
    SELECT *, ROW_NUMBER() OVER (PARTITION BY respondent_id, wave_id
   ORDER BY ingestion_timestamp DESC) AS rn
    FROM research.research_bronze.nat_raw_w14
  ) WHERE rn = 1
) AS s
ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;
```

Deduplicate the source on the merge keys, keeping the latest row, before MERGE. Reference finding 14 in SQL form.

Common wrong answers.

- Line 3: The join condition uses both merge keys. The source is the problem.